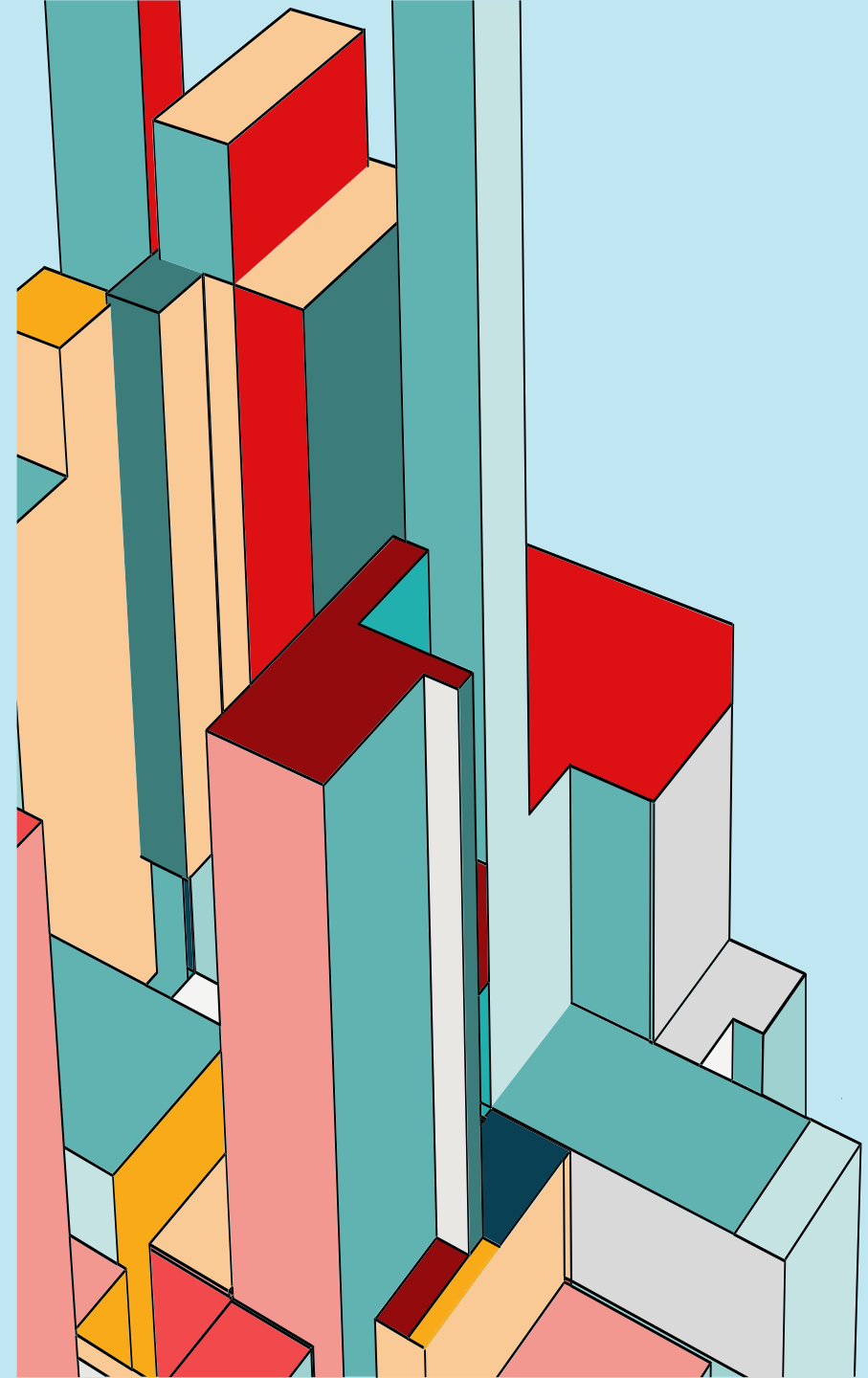


SOFTWARE ARCHITECTURE • arc42

Simple Self-Driving Car using AUTOSAR Adaptive Platform

AUTOSAR Adaptive Platform on Jetson Orin Nano, driving CARLA via
lwrc1 over CycloneDDS + Zenoh



Introduction & goals

§1 · what the architecture must achieve

Closed-loop correctness

Keep the ego vehicle in-lane and stop for obstacles in CARLA. The perception→localization→planning→control loop must actually drive.

AC-01 · AC-02

Bounded, stable timing

Control loop ≥ 20 Hz, end-to-end sense→act ≤ 100 ms. DDS transport adds only microseconds, so compute dominates the budget.

NFR-PERF-01/02

Portability

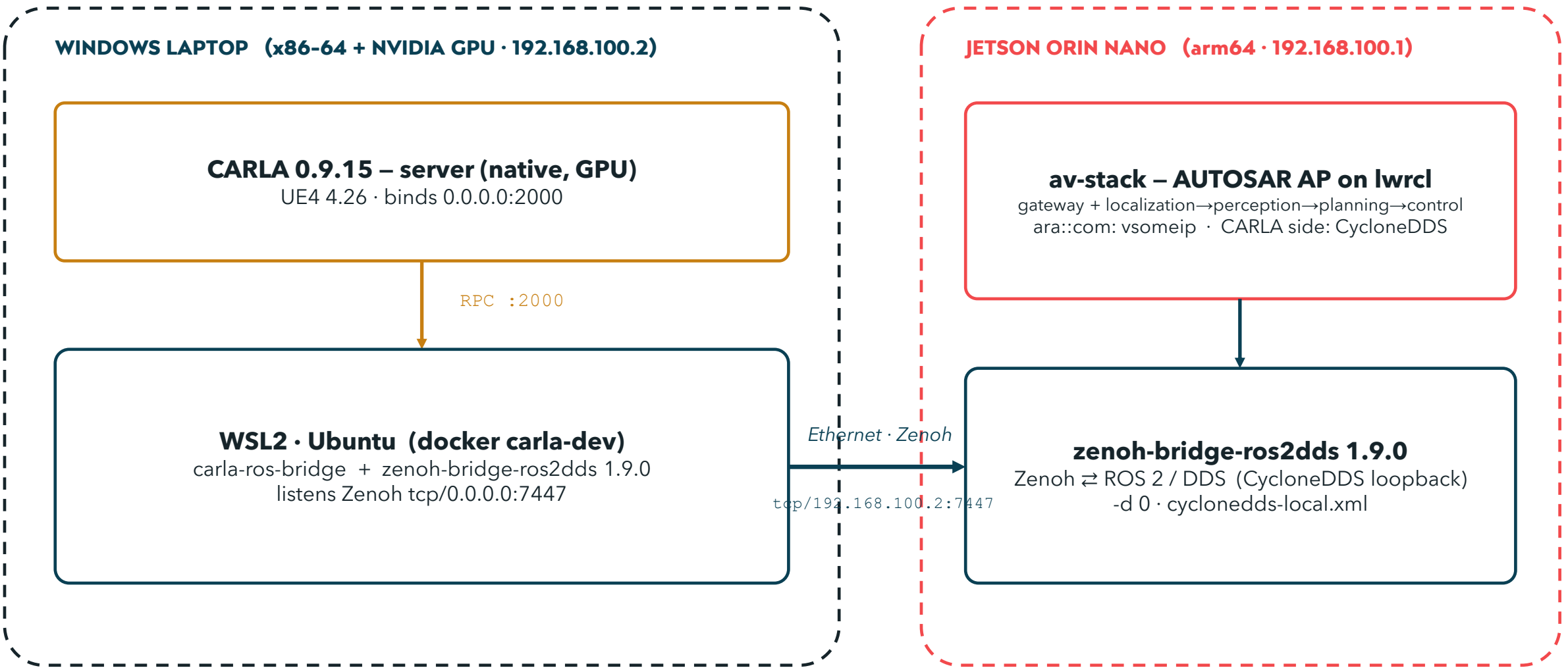
One source set builds on the CycloneDDS backend for bring-up and the adaptive-autosar (ara::com) backend for the Jetson – unchanged.

NFR-PORT-01

Driving forces: reuse the mature carla-ros-bridge · be recognizably AP (ara::exec / sm / phm / log) · small enough to demo on one Jetson.

Context & scope

§3 · the system boundary and its neighbours



The AP side never talks to CARLA directly – the bridge is the sole gateway (ADR-03). All external data crosses Zenoh tcp/7447 · DDS stays local.

Solution strategy

§4 · five moves

1 • Modular apps

Pipeline → independent Adaptive Applications, one process each, ordered by `ara::exec`.

2 • Program once on `lwrcl`

`rlcpp`-compatible source; `cyclonedds` backend for bring-up, `adaptive-autosar` for target.

3 • Reuse CARLA bridge

External system; only a thin Vehicle Interface owns the actuation contract.

4 • AP platform services

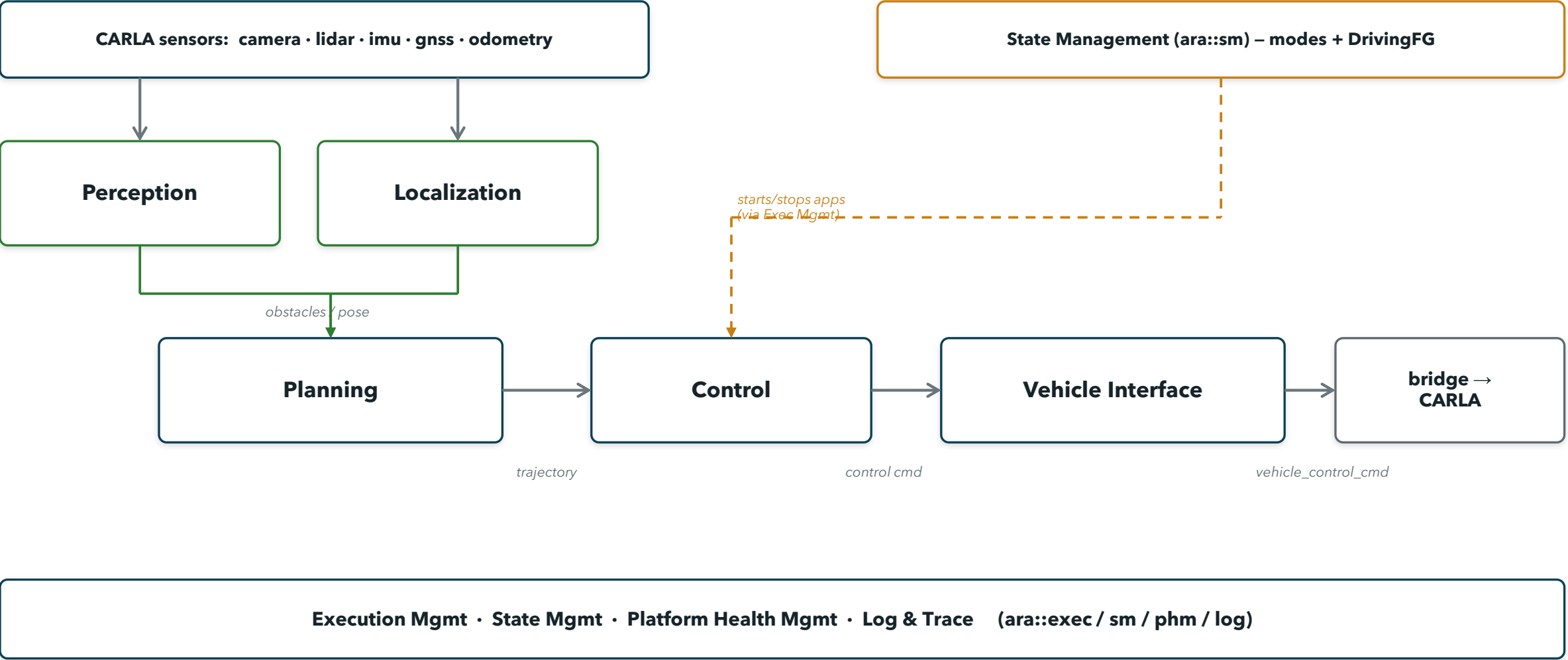
`ara::sm` states, `ara::phm` supervision, `ara::log` observability.

5 • Data-driven config

Gains/thresholds/topics/QoS in YAML + generated ARXML – no code change to retarget.

Building block view

§5 · Adaptive Applications + platform services



Sensor apps subscribe CARLA topics directly; only actuation is gatewayed via Vehicle Interface. pose also feeds Control. All internal topics are timestamped for freshness checks.

Interfaces — what the Jetson receives from CARLA

ROS 2 topics the AP self-driving stack subscribes, via carla-ros-bridge (role_name = ego_vehicle)

ROS 2 topic (/carla/ego_vehicle/...)	ROS 2 message type	Consumed by
.../rgb_front/image	sensor_msgs/msg/Image	Perception – camera
.../lidar	sensor_msgs/msg/PointCloud2	Perception – LiDAR
.../imu	sensor_msgs/msg/Imu	Localization
.../gnss	sensor_msgs/msg/NavSatFix	Localization
.../odometry	nav_msgs/msg/Odometry	Localization
.../vehicle_status	carla_msgs/msg/CarlaEgoVehicleStatus	Vehicle Interface

Publishes back to CARLA: /carla/ego_vehicle/vehicle_control_cmd · carla_msgs/msg/CarlaEgoVehicleControl (throttle / steer / brake, ADR-04)

Same DDS domain, matching QoS (bridge publishes RELIABLE · VOLATILE). Exact sensor topic names follow the bridge sensor config (objects.json); carla_msgs types are generated into lwrcl as IDL.

These are the ROS 2 interfaces at the CARLA boundary – the contract the Jetson AP applications must match to be discovered by the running bridge.

Interfaces — ara::com services (SOME/IP)

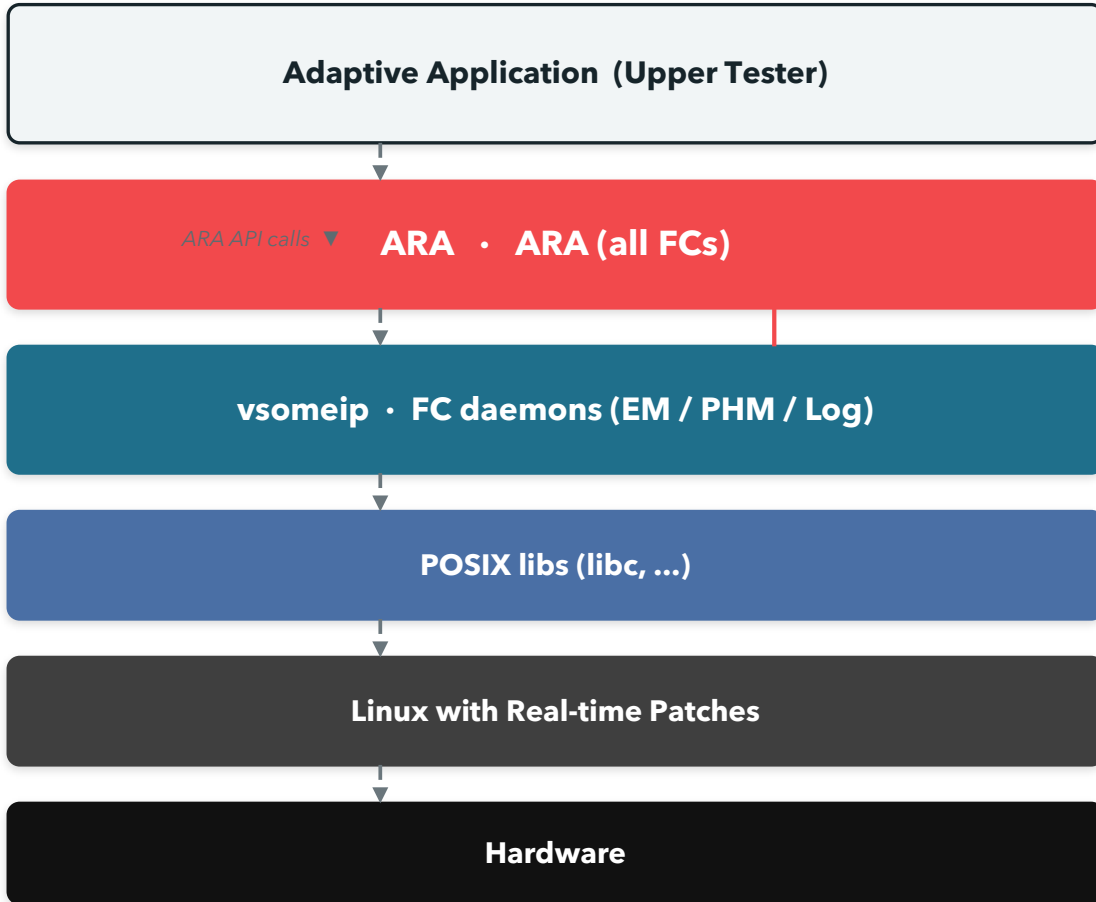
service · instance · event IDs of the internal AA↔AA contract (vsomeip binding)

Service	Svc / Inst	Events (ID)	Provider (skeleton)	Consumers (proxy)	UDP
SensorService	0x1001 / 0x0001	imu 0x8001 · gps 0x8002 · speed 0x8003 · lidar 0x8004	carla_gateway	localization_app · perception_app	30510
VehicleStateService	0x1002 / 0x0001	state 0x8001	localization_app	perception_app · planning_app · control_app	30511
ObjectsService	0x1003 / 0x0001	objects 0x8001	perception_app	planning_app	30512
TrajectoryService	0x1004 / 0x0001	trajectory 0x8001	planning_app	control_app	30513
ControlService	0x1005 / 0x0001	cmd 0x8001	control_app (safe_stop_app in SafeStop)	carla_gateway	30514

App IDs: gateway 0x1101 · localization 0x1102 · perception 0x1103 · planning 0x1104 · control 0x1105 · safe_stop 0x1107; routing = autosar_vsomeip_routing_manager (0x1000); SOMEIP

ARA Functional-Cluster interactions

the layered ECU stack – one slide per used FC follows



HOW TO READ THESE SLIDES

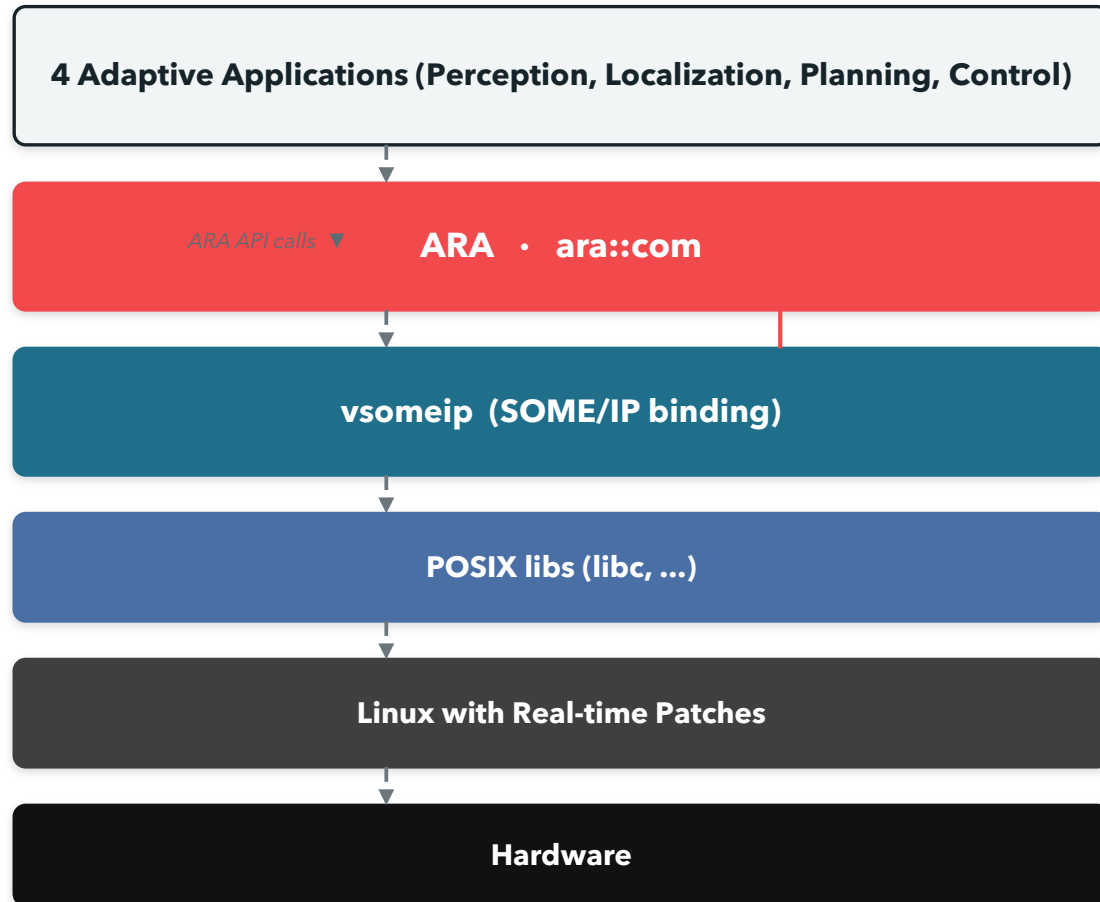
Same layering as the AUTOSAR System Tests
ECU test setup (App -> ARA -> binding ->
POSIX -> Linux+RT -> Hardware).

Dashed down = the AA calls the ARA::FC API,
which drives the binding and OS below.
Red up = callbacks/events flow back to the AA.

The next four slides detail ara::com, ara::exec,
ara::phm and ara::log as used by av-stack.

FC interaction — ara::com

communication – service-oriented data exchange



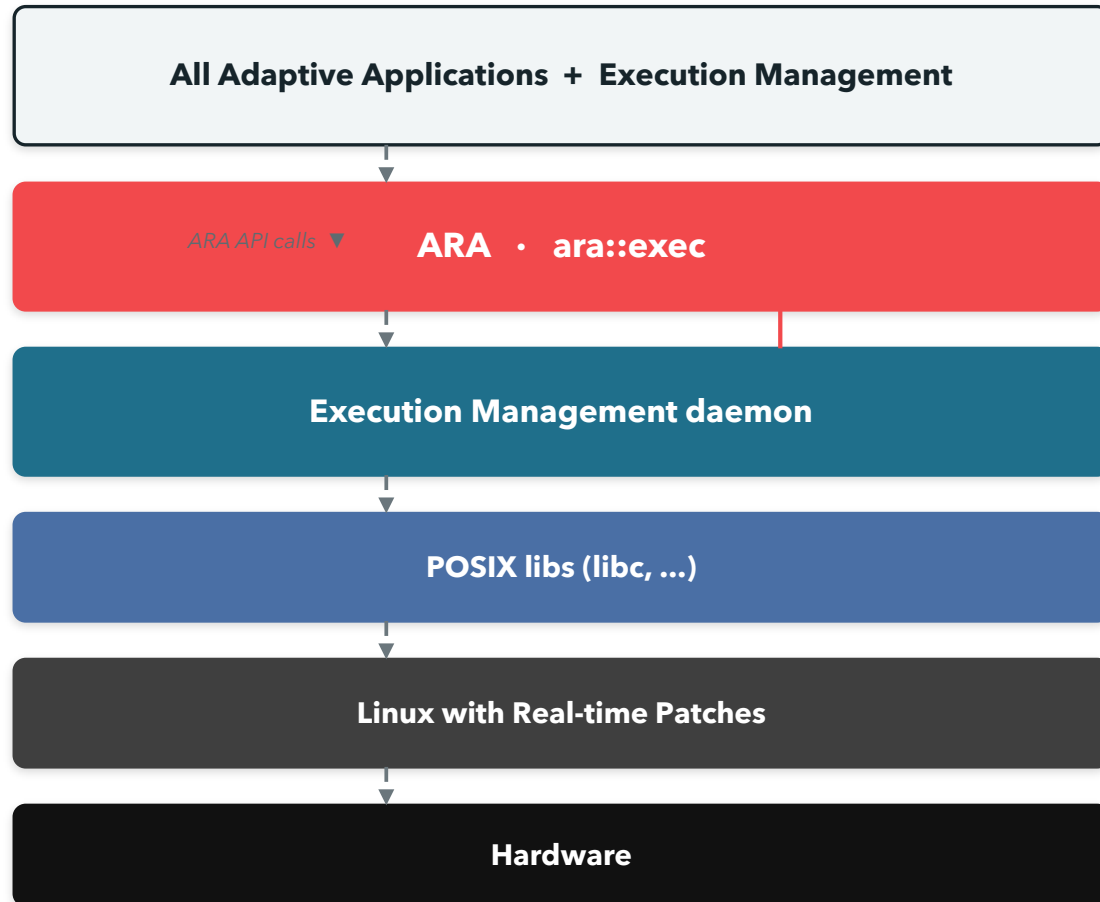
INTERACTION (AA $\rightleftharpoons$ ARA::FC $\rightleftharpoons$ LOWER LAYERS)

AA -> ara::com: FindService, OfferService, Subscribe, SetReceiveHandler, Send(sample)
ara::com: proxy/skeleton, (de)serialize, service discovery, event dispatch, QoS
vsomeip: SOME/IP messages; POSIX sockets -> Linux net stack -> NIC (or shared memory)
Back to AA: receive handler + GetNewSamples

Validated: host test_soa_pipeline (4 AAs over ara::com); STS_CM_* in AUTOSAR System Tests

FC interaction — ara::exec

execution management – process lifecycle



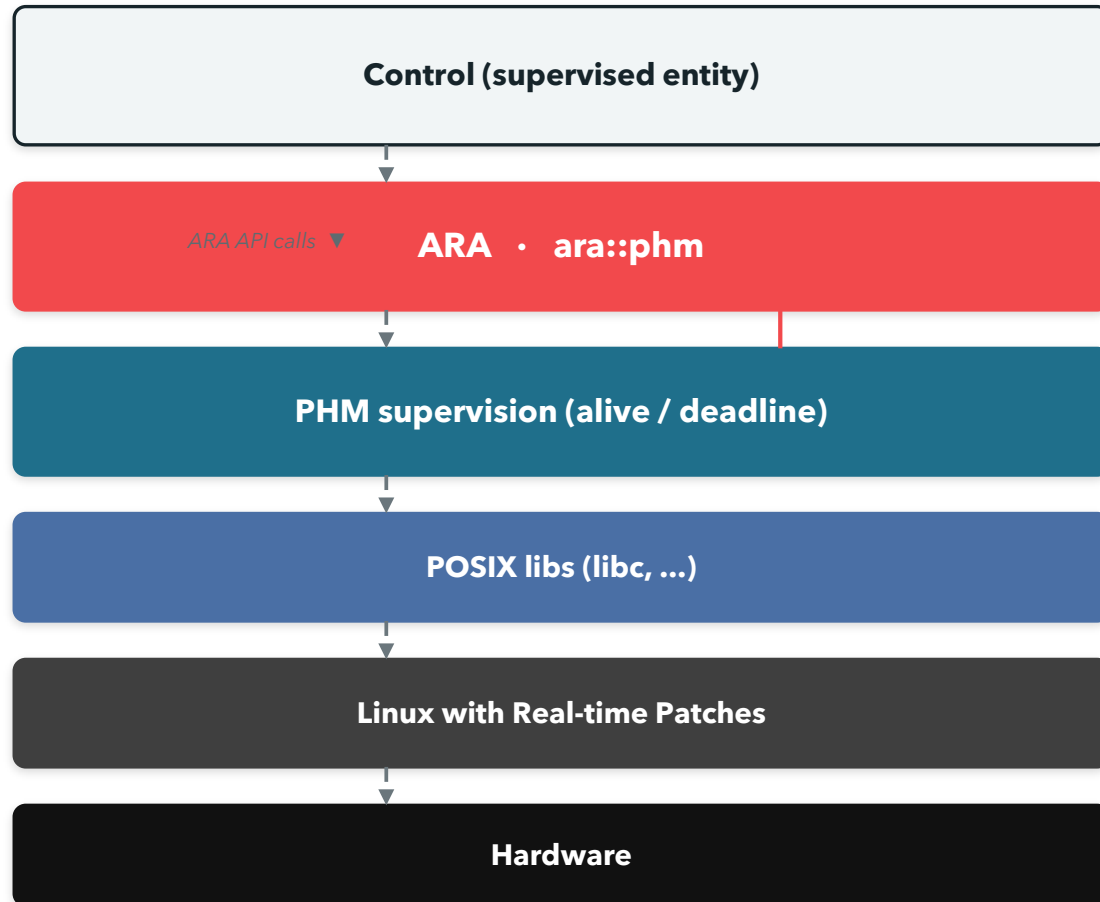
INTERACTION (AA ⇄ ARA::FC ⇄ LOWER LAYERS)

EM starts one process per app from the
Execution Manifest when AvStack = Driving
AA -> ara::exec: ReportExecutionState(kRunning)
EM: POSIX fork/exec, scheduling, resource groups,
ordered start via EXECUTION-DEPENDENCY
POSIX -> Linux RT scheduler -> CPU cores
Back to AA: state change / restart on fault

Validated: exec lifecycle in test_ap_integration;
STS_EMO_* in AUTOSAR System Tests

FC interaction — ara::phm

platform health management – supervision



INTERACTION (AA ⇔ ARA::FC ⇔ LOWER LAYERS)

AA -> ara::phm: `SupervisedEntity.ReportCheckpoint`

ara::phm: alive + deadline supervision of the control loop (checkpoints vs deadlines)

Recovery: request State Management to switch

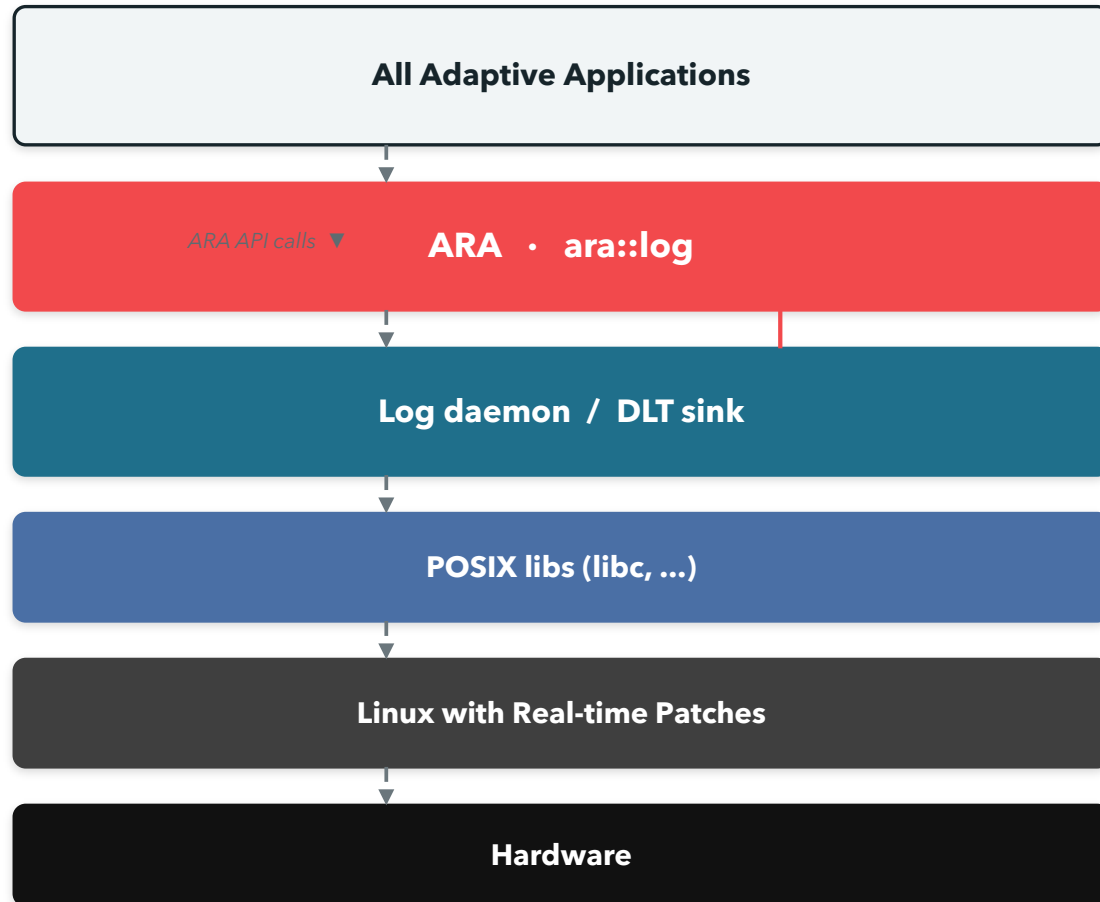
`DrivingFG Active -> SafeStop`; Execution Management stops `control_app` and starts `safe_stop_app` (brake)

Note: Jetson Orin Nano has only a SoC-internal watchdog (`/dev/watchdog`); no external safety watchdog

Validated: safe-stop path (UC-4); PHM RS in Tests

FC interaction — ara::log

log and trace – diagnostics output



INTERACTION (AA ⇄ ARA::FC ⇄ LOWER LAYERS)

AA -> ara::log: `Logger.LogInfo()/LogWarn()/LogError()` (one line per app event)

ara::log: severity-tagged, context-tagged records

DLT / console sink: POSIX write / socket

Linux -> file or network (off-board log client)

Back to AA: none (one-way)

Validated: [INFO] lines in host runs (offer/subscribe traces); STS_LT_* in System Tests

Vehicle modes & State Management

ara::sm Machine States + Function Groups decide which apps run (ModeDependentStartupConfig)

Machine States (vehicle modes)

Startup

Driving

Parking

IgnitionOff

Function Group DrivingFG

Off

Active

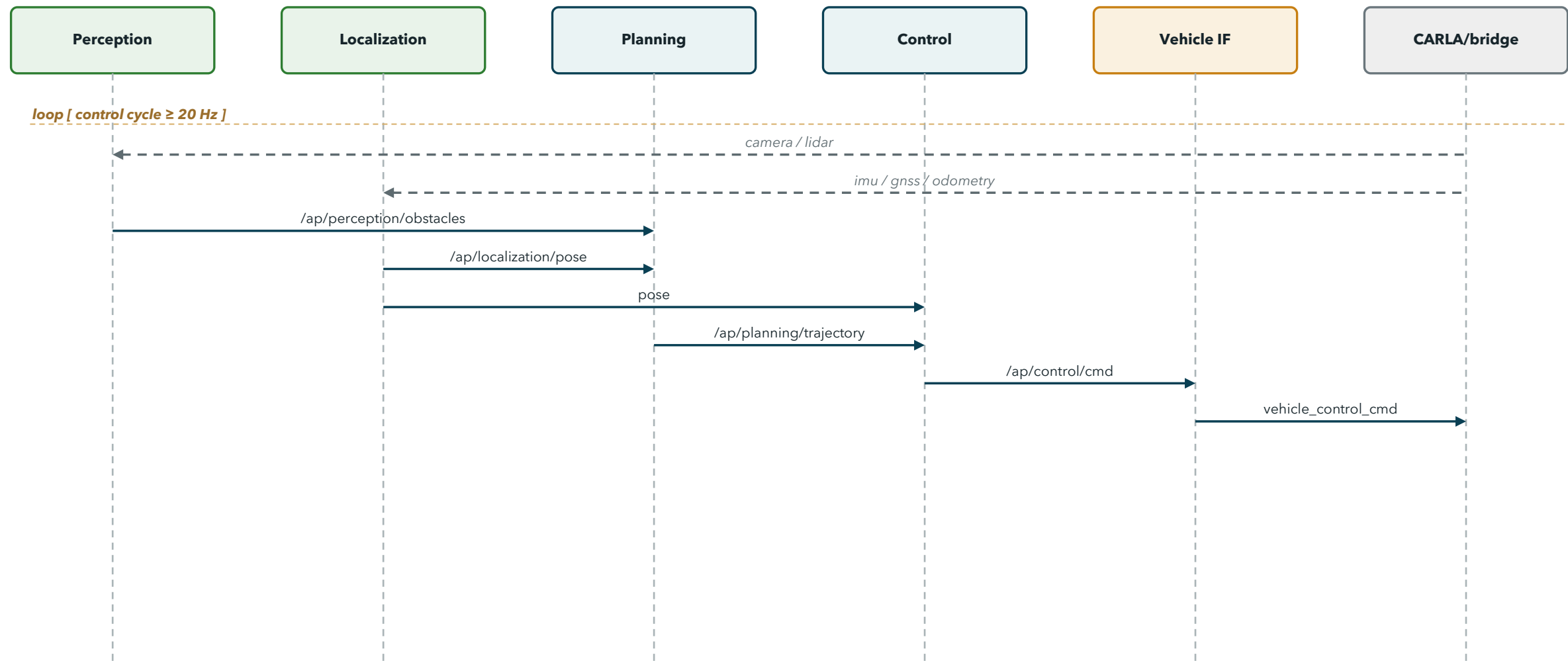
SafeStop

Application	Bound to	Startup state
carla_gateway	machineMode	Driving
localization_app	machineMode	Driving
perception_app	functionGroup	DrivingFG · Active
planning_app	functionGroup	DrivingFG · Active
control_app	functionGroup	DrivingFG · Active
safe_stop_app	functionGroup	DrivingFG · SafeStop

State Management sets the mode; Execution Management starts/stops each process per its ModeDependentStartupConfig. PHM fault: DrivingFG Active → SafeStop (control stops, safe_stop brakes). No custom state service.
Realization status: modes & process bindings live in av_machine/execution_manifest.arxml; SM/EM daemons are not deployed yet – run_ap.sh stands in (accepted debt, §11).
Modes per AUTOSAR_AP_TR_SystemTests §17.1 (Machine Manifest / Execution Manifest) · av-stack

Runtime – nominal driving loop

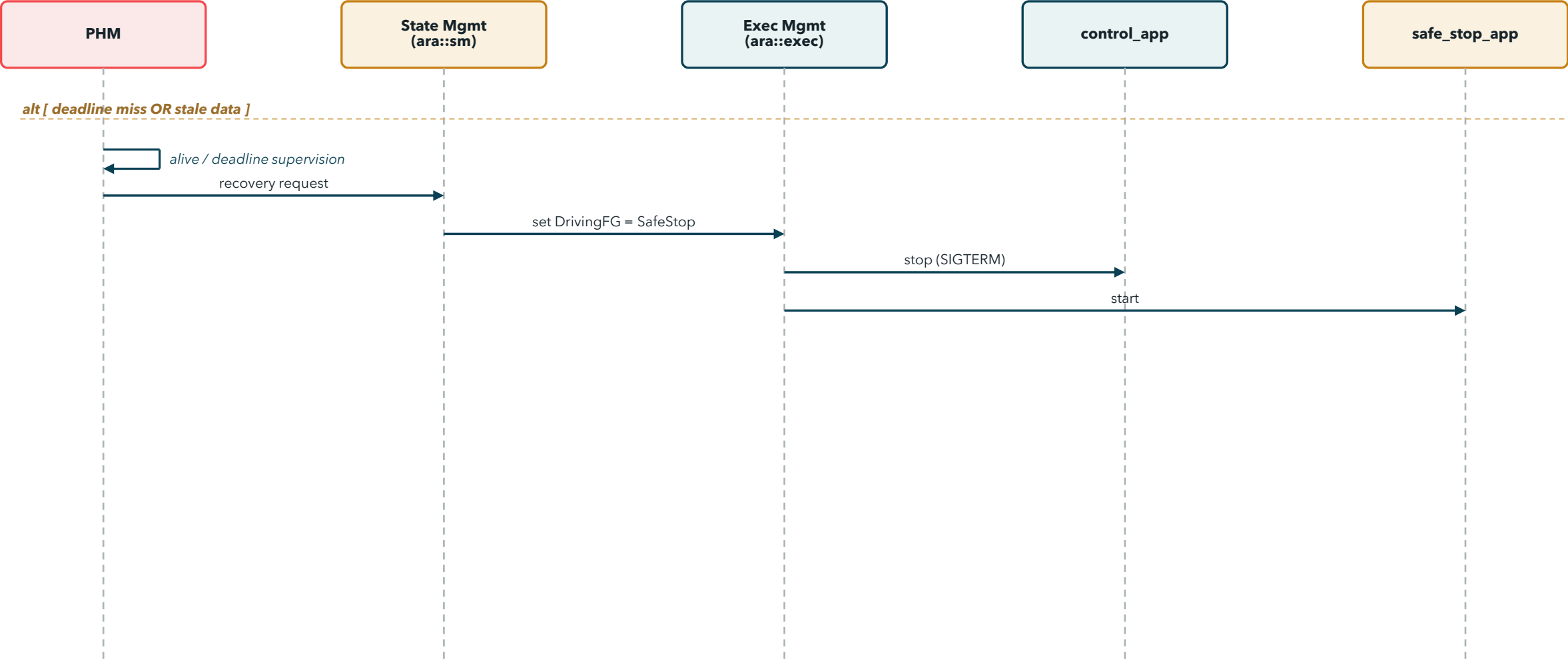
§6 · RS-2 · steady-state control cycle



Control runs at loop rate even if Perception is slower (async); consumers reject stale samples via timestamp – sense→act ≤ 100 ms.

Runtime – safe-stop on fault

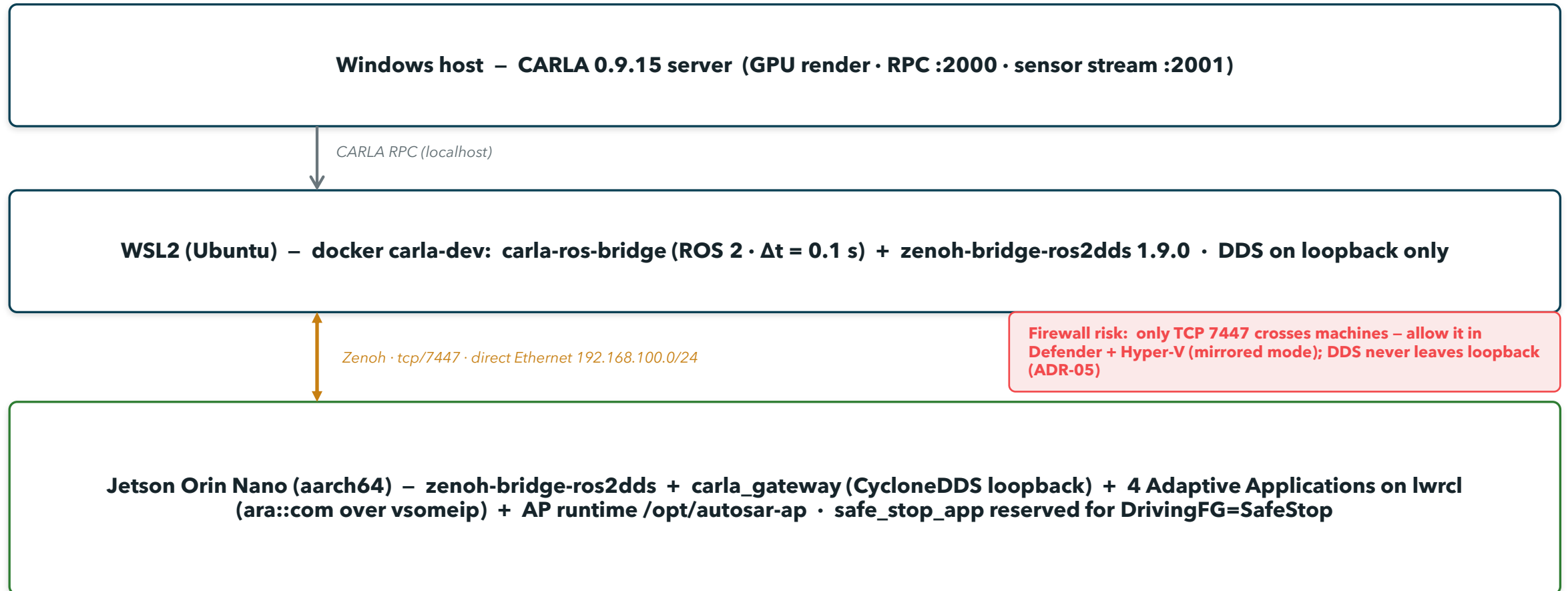
§6 · RS-4 · supervision-driven degrade (ara::sm + ara::exec)



State Management changes the function-group state; Execution Management stops control_app and starts safe_stop_app (which commands brake). No in-app state gating. Realization status: modes & process bindings live in av_machine/execution_manifest.xml; SM/EM daemons are not deployed yet – run_ap.sh stands in (accepted debt, §11).

Deployment view

§7 · nodes and the Zenoh + CycloneDDS data path



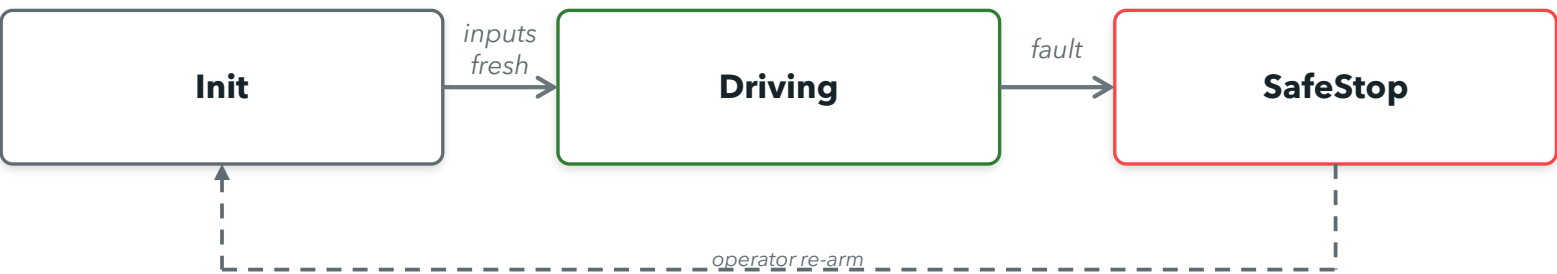
Internal AA↔AA binding = ARA_COM_EVENT_BINDING=vsomeip (SOME/IP); the gateway's CARLA side = CycloneDDS via cyclonedds-local.xml.

AP SDC arc42 architecture · lwrc1 · CARLA ros-bridge · AUTOSAR AP R24-11

Crosscutting concepts

§8 · state, health, time & portability

System state machine (ara::sm)



Health & supervision

PHM alive + deadline checks on the control loop; a violation drives State Mgmt to SafeStop.

`ara::phm`

Time & freshness

One time base stamps every header; consumers reject stale samples – the uniform guard against a dead upstream.

`ara::tsync / Clock`

Portability

Identical source on cyclonedds (bring-up) and adaptive-autosar (target). Transport via env var; config in YAML/ARXML.

NFR-PORT-01

Architecture decisions

§9 · ADR-01...07

ADR-01 lwrl backend

Where rclcpp-compatible; adaptive-autosar on target

Roles one codebase, ROS interop, no ROS 2

ADR-02 vsomeip AA binding

Where ARA_COM_EVENT_BINDING=vsomeip

Roles AP-native SOA; DDS at gateway only

ADR-03 Reuse CARLA bridge

Where CARLA+bridge external; DDS only

Roles big scope reduction

ADR-04 CarlaEgoVehicleControl

Where direct throttle/steer/brake egress

Roles no extra bridge node

ADR-05 Zenoh cross-machine

Where 2× zenoh-bridge-ros2dds · tcp/7447

Roles one TCP port through WSL2/firewall

ADR-06 One process per app

Where started/ordered by ara::exec

Roles isolation + resource limits

ADR-07 EKF localizer

Where EKF in code; GT odometry fallback

Roles fastest path to closed loop

Risks & technical debt

§11 · what could bite, and what we deferred

R1 Zenoh link (7447)

Where no data flow if it fails

Roles match bridge versions; `nc -vz first`

R2 Perception latency

Where loop slows on Orin Nano

Roles `async`; `TensorRT`; `downscale`

R3 QoS mismatch

Where silent no-subscription

Roles match QoS per IF-QoS

R4 AP codegen setup

Where build blocked

Roles bring up on cyclonedds first

R5 carla_msgs drift

Where type incompatibility

Roles `pin bridge`; `gen IDL from defs`

R6 Clock offset

Where false stale-data stops

Roles common time base; tolerant thresholds

Accepted debt: minimal perception, manual safe-stop recovery, illustrative (uncertified) PHM, single ego/map.

AP SDC arc42 architecture · `lwrc1` · CARLA ros-bridge · AUTOSAR AP R24-11

Build & run — host verification

Path A · no hardware · the SOA loop is verified this way

```
# cores + ara::com SOA wiring — no lwrcl, no ROS
./build_ap.sh host          # cmake build + ctest -> 100% tests passed

# or directly with g++ (no cmake):
g++ -std=c++17 -Isrc/av_ap/include -Isrc/av_common/include \
    -Isrc/av_perception/include -Isrc/av_localization/include \
    -Isrc/av_planning/include -Isrc/av_control/include \
    src/av_integration_tests/test/test_soa_pipeline.cpp \
    src/av_perception/src/perception_core.cpp src/av_localization/src/ekf_localizer.cpp \
    src/av_planning/src/planner_core.cpp src/av_control/src/controller_core.cpp \
    -o test_soa && ./test_soa
```

Expected: final: x=96.62 m, v=0.000 m/s, detections=331

PASS test_soa_pipeline: SOA loop over ara::com (DrivingFG=Active) stopped 3.4 m before obstacle

Four Adaptive Applications wired through ara::com only; an in-process broker stands in for the SOME/IP (vsomeip) binding.

AP SDC arc42 architecture · lwrcl · CARLA ros-bridge · AUTOSAR AP R24-11

Build & run — Jetson ⇔ CARLA demo

Path B · full stack procedure (needs lwrcl + hardware)

1 • Windows – CARLA

Launch CARLA 0.9.15; RPC :2000, sensor stream :2001.

```
CarlaUE4.exe -quality-level=Low
```

2 • WSL2 – bridge (docker)

In container carla-dev: source-built carla-ros-bridge (0.9.15 pin); sim tick must match sensor_tick.

```
fixed_delta_seconds:=0.1  
objects_definition_file:=.../objects.json
```

3 • Network – Zenoh

Mirrored networking; only TCP 7447 crosses machines (Defender + Hyper-V rules). DDS stays on loopback.

```
nc -vz 192.168.100.2 7447
```

4 • Jetson – build

CycloneDDS + vsomeip; AP runtime /opt/autosar-ap; lwrcl adaptive-autosar; add carla_msgs IDL.

```
./build_ap.sh adaptive-autosar
```

5 • Run

vsomeip routing manager, then gateway -> localization -> perception -> planning -> control.

```
ARA_COM_EVENT_BINDING=vsomeip ./run_ap.sh
```

6 • Verify

Car drives & stops for the obstacle; lidar ~10 Hz; kill gateway -> safe-stop.

```
ros2 topic hz /carla/ego_vehicle/lidar
```

Internal AA↔AA = ara::com over SOME/IP (vsomeip); CARLA boundary = CycloneDDS loopback + Zenoh via the lwrcl gateway. Full detail in BUILD_AND_RUN.md.

AP SDC arc42 architecture · lwrcl · CARLA ros-bridge · AUTOSAR AP R24-11

Build & run — field notes

ops · issues met during WSL ⇌ Jetson bring-up, and their fixes

1 • Frame drops every tick

fixed_delta_seconds must equal camera/lidar sensor_tick (0.1 s). One warning at startup + rare GNSS lates are benign.

```
fixed_delta_seconds:=0.1
```

2 • Bridge rejects 0.9.15

Version pin, not an API break – update CARLA_VERSION and rebuild; keep pip carla==0.9.15 (match the server).

```
echo 0.9.15 > ../carla_ros_bridge/CARLA_VERSION
```

3 • Humble build breaks

tf2_eigen .h → .hpp rename on Ubuntu 22.04; skip the unneeded packages instead of patching.

```
touch {pcl_recorder,rviz_carla_plugin}/COLCON_IGNORE
```

4 • topic hz silent

If echo --once works, frames are sparse, not missing. Stale ros2 daemon or env mismatch mimics this.

```
ros2 daemon stop · ros2 topic info --verbose
```

5 • Big lidar clouds drop

Raise kernel UDP buffers to 16 MB on the WSL host – docker containers share the host kernel.

```
sysctl net.core.rmem_max=16777216
```

6 • Zenoh bridges won't peer

Same zenoh-bridge-ros2dds version on both sides (1.9.0); container needs --network host; port 7447 open.

```
zenoh-bridge-ros2dds --version
```

Validation — bench demo

the physical rig running the full chain, live

1 • CARLA 0.9.15 – Windows laptop

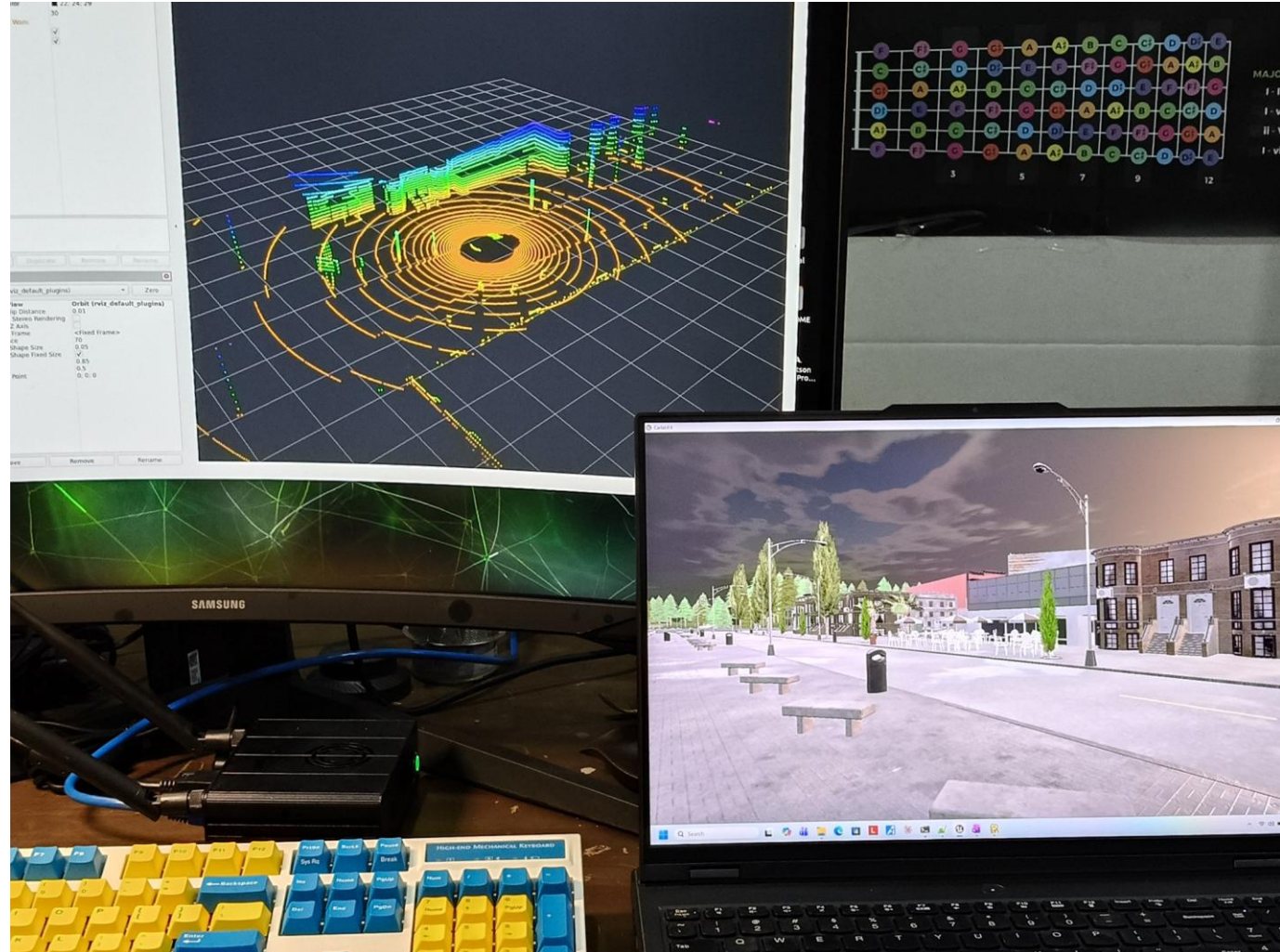
Town square rendered live (CarlaUE4); ego + av-stack sensor set spawned by the dockerised carla-ros-bridge.

2 • Lidar in RViz – after the Zenoh hop

/carla/ego_vehicle/lidar point cloud (32 ch, ~10 Hz): rings + buildings visible = sensor path CARLA → bridge → Zenoh → Jetson verified.

3 • Jetson Orin Nano – av-stack

The black box on the desk: carla_gateway + 4 AAs over vsomeip, CycloneDDS on loopback, zenoh-bridge to the laptop via the direct cable.



Live evidence for the sensing path (AC-02 data flow); closed-loop driving & stop (AC-01) is demonstrated with the same rig – see Build & run, Path B.

References

standards, simulators and the upstream projects this stack builds on

AUTOSAR – Adaptive Platform standard (R24-11): SWS/EXP for ara::com, exec, sm, phm, log autosar.org

CARLA Simulator – open-source driving simulator (UE4); server 0.9.15 on the Windows host carla.org

Autoware Foundation – open-source AD stack; origin of the replaced ROS 2 pipeline autoware.org

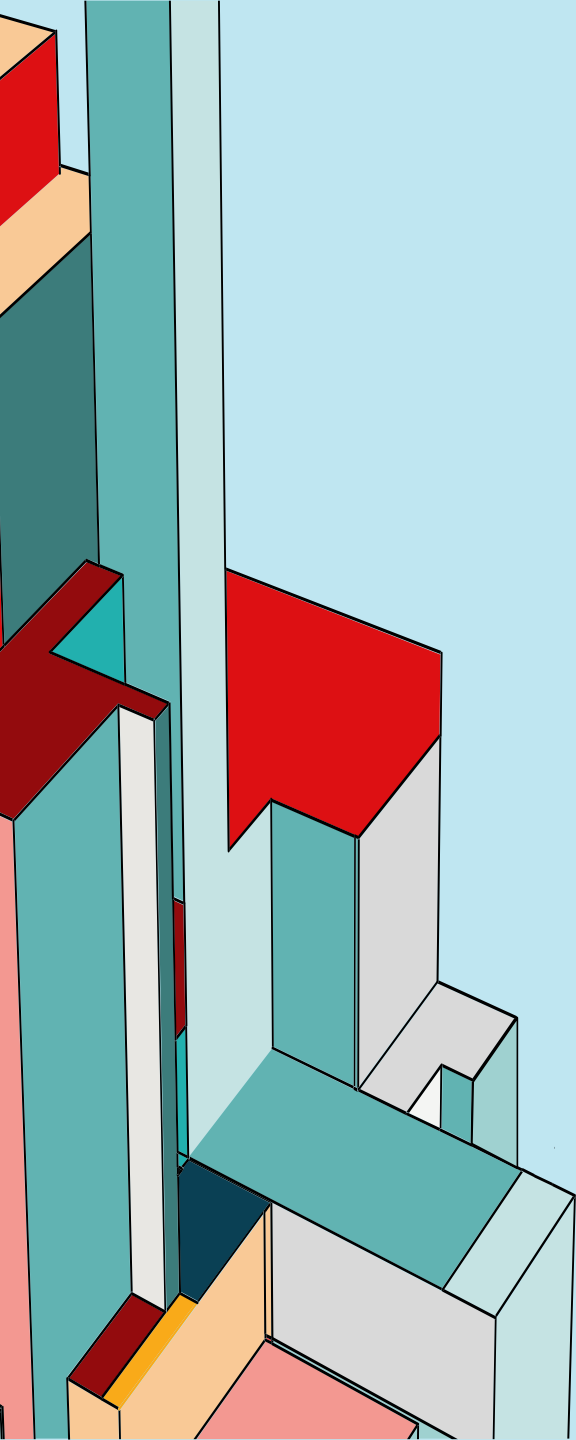
Udacity Self-Driving Car – course material behind the perception / localization / planning / control concepts github.com/udacity

tatsuyai713/Adaptive-AUTOSAR – Adaptive Runtime AUTOSAR Linux simulator – the AP runtime on the Jetson
github.com/tatsuyai713/Adaptive-AUTOSAR

tatsuyai713/lwrcl – LightWeight Rclcpp Compatible Library – rclcpp API without ROS 2 github.com/tatsuyai713/lwrcl

evshary/autoware_carla_launch – ROS 2 launch for Autoware + CARLA; source of the Zenoh bridge setup
github.com/evshary/autoware_carla_launch

eclipse-zenoh/zenoh-plugin-ros2dds – zenoh-bridge-ros2dds – the cross-machine transport (tcp/7447) github.com/eclipse-zenoh/zenoh-plugin-ros2dds



Thank you

[Nguyen Nguyen · baonguyenpro@gmail.com](mailto:baonguyenpro@gmail.com)